

Trunk-Based Development

트렁크 기반 개발 전략 및 실천 가이드

Git Flow와의 비교 분석과 CI/CD 연계 방법론

SECTION 01

학습 목표

이 세션의 목표

Learning Objectives

이번 세션을 통해 Trunk-Based Development의 핵심 개념을 이해하고 실무에 적용할 수 있는 구체적인 방법론을 학습합니다.



CONCEPT

TBD 개념 이해

Trunk-Based Development의 정의와 핵심 원칙, 그리고 왜 현대적 개발에서 중요한지 학습합니다.



COMPARISON

Git Flow vs TBD

전통적인 Git Flow 방식과의 차이점을 비교하고, TBD가 가지는 병합 충돌 최소화 이점을 분석합니다.



PRACTICE

TBD 실천 방법

짧은 생명주기의 브랜치 관리, Feature Flags 활용 등 구체적인 실천 가이드를 익힙니다.



INTEGRATION

CI/CD 연계

지속적 통합(CI) 파이프라인과 TBD가 어떻게 상호 보완하며 배포 속도를 높이는지 이해합니다.



CULTURE

팀 협업 강화

작은 단위의 코드 리뷰와 투명한 진행 상황 공유를 통해 팀 협업 문화를 개선하는 방법을 배웁니다.



SECTION 02

Trunk-Based Development란?

정의와 핵심 원칙

TBD 정의와 핵심 원칙

Trunk-Based Development는

협업의 복잡성을 줄이고

배포 속도를 높이는 전략입니다.

Trunk-Based Development (TBD):

모든 개발자가 단일 브랜치(trunk/main)에 작은 변경사항을 자주 병합하는 개발 방식

"Commit early, commit often"

Integration hell을 방지하는 최선의 방법

CORE PRINCIPLES



단일 메인 브랜치 (Single Trunk)

모든 코드는 'main' 또는 'trunk'라는 단 하나의 공유 브랜치를 중심으로 관리됩니다. 장기 유지되는 개발용 브랜치(develop)를 두지 않습니다.



짧은 생명주기 브랜치

기능 브랜치는 생성 후 몇 시간, 늦어도 24시간 이내에 메인 브랜치로 병합되어야 합니다. 브랜치가 오래될수록 병합 충돌 비용이 기하급수적으로 증가합니다.



작은 커밋, 자주 통합

거대한 변경사항을 한 번에 병합하는 대신, 작고 독립적인 단위로 쪼개어 하루에도 여러 번 통합합니다.



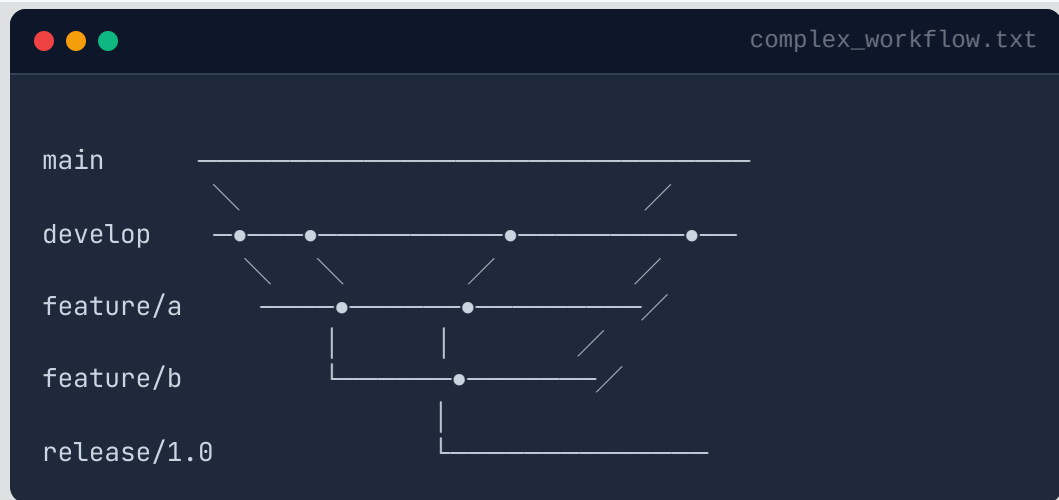
Feature Flags 활용

미완성 기능이 메인 브랜치에 병합되더라도 사용자에게 노출되지 않도록 '플래그(Toggle)'로 감싸서 배포합니다.

전통적 Git Flow vs TBD

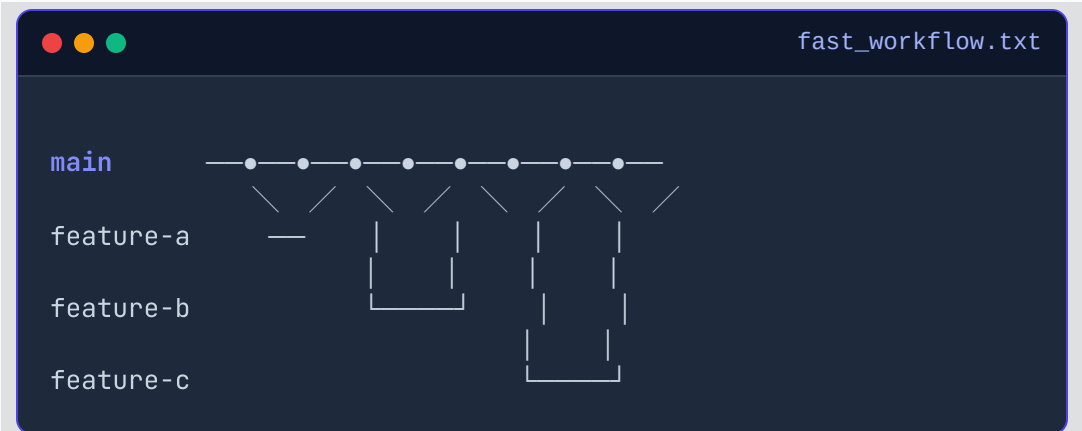


Git Flow



- ❗ **장기 브랜치:** develop, release 등 관리가 필요한 브랜치가 다수 존재
- ❗ **병합 복잡도:** 브랜치 수명이 길어 병합 시 충돌(Conflict) 발생 빈도 높음
- ❗ **통합 지연:** 기능 개발 완료 후 병합까지 시간이 오래 소요됨 ("Integration Hell")

Trunk-Based Development



- ✓ **단일 브랜치:** 오직 'main' 브랜치 하나만 진실 공급원(Source of Truth)으로 관리
- ✓ **짧은 생명주기:** Feature 브랜치는 1일 이내에 병합 및 삭제 (< 24h)
- ✓ **빠른 통합:** 작은 단위로 자주 병합하여 충돌 최소화 및 즉각적인 피드백



SECTION 03

TBD의 이점

핵심 이점

TBD의 이점



전통적 방식보다 압도적으로 빠른
피드백 루프를 제공합니다.

KEY BENEFITS



빠른 피드백 (Rapid Feedback)

변경 사항을 병합하자마자 CI 파이프라인이 즉시 검증합니다. 버그를 초기에 발견하여 수정 비용을 획기적으로 낮춥니다.



병합 충돌 최소화

브랜치 수명이 짧아(하루 미만) 다른 개발자와의 변경 사항 차이(drift)가 적습니다. 거대한 'Merge Hell'을 원천 차단합니다.



진정한 지속적 통합 (CI)

'통합'은 단순히 코드를 합치는 것이 아닙니다. 하루에 여러 번 Main에 병합함으로써 소프트웨어는 항상 배포 가능한 상태를 유지합니다.



팀 협업 및 투명성 향상

모두가 같은 브랜치를 바라보며 작업하므로 진행 상황이 투명합니다. 변경 단위가 작아 코드 리뷰(PR)가 빠르고 부담이 없습니다.

feedback_loop_comparison.txt

// 전통적 방식 (Git Flow)

Feature 개발 (3주)

→ 병합 충돌 해결 (1주)

→ QA 테스트 (1주)

= 5주 후 피드백 🐢

// Trunk-Based Development

작은 변경 (1일)

→ 즉시 병합 (Main)

→ CI 자동 테스트 (10분)

= 1일 후 피드백 🚀



SECTION 04

TBD 실천 방법

작게, 자주, 자동화

작은 커밋과 짧은 브랜치



TBD의 핵심은 **작업 단위를 최소화**하여 통합 리스크를 줄이는 것입니다.

전략 1: 작은 커밋 (Atomic Commits)

거대한 변경사항 대신 논리적으로 완결된 최소 단위로 쪼개어 커밋합니다.

```
bash (terminal)

# ❌ 나쁜 예: 일주일치 작업을 한 번에 커밋
git commit -m "Implement entire user auth system"

# ✅ 좋은 예: 작업 단위 분리 (자주 통합)

# 1. 모델 추가
git commit -m "Add user model structure"
git push origin main

# 2. API 엔드포인트 구현
git commit -m "Add login API endpoint"
git push origin main

# 3. 토큰 생성 로직
git commit -m "Add JWT token generation"
git push origin main
```

전략 2: 짧은 생명주기 (< 24시간)

브랜치 수명은 하루를 넘기지 않습니다. 퇴근 전까지 병합하거나 삭제합니다.

```
workflow.sh

# [09:00] 브랜치 생성 및 작업 시작
git checkout -b feature/user-avatar
git commit -m "Add avatar upload UI"

# [13:00] PR 생성 (작은 변경사항 → 빠른 리뷰)
git push origin feature/user-avatar

# GitHub에서 Pull Request 생성...

# [15:00] 리뷰 승인 후 즉시 병합 및 동기화
git checkout main
git pull origin main

# [15:10] 브랜치 삭제 (생명주기 종료)
git branch -d feature/user-avatar

# 다음 작업을 위해 새로운 짧은 브랜치 시작
```

Feature Flags & Automated Testing



Feature Flags

배포(Deploy)와 출시(Release)를 분리

```
UserDashboard.js

// 미완성 기능을 Feature Flag로 숨겨서 관리
function UserDashboard() {
  const showNewUI = featureFlags.isEnabled('new-dashboard');

  if (showNewUI) {
    return <NewDashboard />; // 개발 중인 기능
  }

  return <OldDashboard />; // 현재 운영 버전
}
```



Automated Testing (CI)

모든 커밋에 대해 자동으로 정합성 검증

```
.github/workflows/ci.yml

name: CI Pipeline
on:
  push:
    branches: ["main"] # 메인 브랜치 푸시 시 실행

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run Unit Tests
        run: npm test

      - name: Block merge on failure
        if: failure()
        run: exit 1
```



SECTION 05

TBD 워크플로우

팀 규모별 워크플로우

TBD 워크플로우 비교



소규모 팀 워크플로우

1-10명 규모 • 빠른 속도 중시

```
terminal - small_team

# 1. 최신 main 가져오기
$ git checkout main
$ git pull origin main

# 2. 직접 커밋 또는 초단기 브랜치
$ git add .
$ git commit -m "Fix login typo"

# 3. 즉시 Push (사후 리뷰 가능)
$ git push origin main

# 완료! 별도 PR 절차 생략 가능
```

KEY FEATURES

- **절차 최소화:** 신뢰 기반의 직접 커밋 허용
- **즉시 배포:** 코드가 main에 도달하는 시간 < 10분
- **Pair Programming:** 코드를 작성하며 실시간 리뷰



중대형 팀 워크플로우

10명+ 규모 • 안정성/동기화 중시

```
terminal - large_team

# 1. 짧은 Feature 브랜치 생성
$ git checkout -b feature/pagination

# 2. 작업 후 Main과 Rebase (동기화)
$ git fetch origin main
$ git rebase origin/main

# 3. Push & PR 생성 (CI 자동 실행)
$ git push origin feature/pagination
$ gh pr create --fill

# CI 통과 + 승인 1개 → 즉시 병합
```

KEY FEATURES

- **Short-Lived Branches:** 24시간 이내 삭제 원칙
- **Rebase 필수:** 선형적 히스토리 유지 (No Merge Commit)
- **CI Gate:** 테스트 실패 시 병합 원천 차단



SECTION 06

TBD 성공 전략

안전하고 점진적으로



06

TBD 성공 전략: Branch by Abstraction



대규모 리팩토링이나 교체 작업을 장기 브랜치 없이 메인에서 수행하는 기법

```
payment_migration.js

// Step 1: 기존 레거시 코드 (직접 호출)
function processPayment(data) {
  return oldPaymentService.process(data);
}

// Step 2: 추상화 레이어 추가 (Indirection)
function processPayment(data) {
  const service = getPaymentService();
  return service.process(data);
}

// Step 4: Feature Flag로 분기 처리
function getPaymentService() {
  if (featureFlags.isEnabled('new-payment')) {
    return newPaymentService;
  }
  return oldPaymentService; // 기본값
}

// Step 6: 완료 후 정리 (Flag 제거)
function processPayment(data) {
  return newPaymentService.process(data);
}
```

Step 1

기존 코드 분석

교체할 레거시 코드의 의존성을 파악하고 호출 지점을 확인합니다.

Step 2

추상화 레이어 도입

직접 호출을 인터페이스나 함수로 감싸서 추상화합니다. (Main 병합)

Step 3

새 서비스 구현

추상화된 인터페이스에 맞춰 새로운 로직을 구현합니다. (Main 병합)

Step 4

Feature Flag 전환

플래그 상태에 따라 구/신 구현체를 선택하도록 코드를 수정합니다.

Step 5

점진적 롤아웃

10% → 50% → 100% 순으로 트래픽을 새 구현체로 이동시킵니다.

Step 6

최종 정리 (Cleanup)

Feature Flag와 추상화 레이어, 레거시 코드를 모두 삭제합니다.

TBD 성공 전략: 고급 기법



Dark Launching

새 기능을 프로덕션에 배포하되 사용자에게 UI를 노출하지 않고, **백그라운드에서 실행**하여 성능과 안정성을 검증하는 기법입니다.

```
searchService.js

function search(query) {
  // 1. 기존 안정된 로직 실행 (사용자 반환용)
  const result = oldSearchEngine.search(query);

  // 2. 새 엔진 백그라운드 테스트 (사용자 모르게)
  if (featureFlags.darkLaunch('new-search')) {
    newSearchEngine.search(query)
      .then(newResult => {
        // 결과 비교 및 로깅 (오류가 있어도 무방)
        logComparison(result, newResult);
      })
      .catch(err => logger.silentError(err));
  }

  return result; // 항상 기존 결과 반환
}
```



Incremental Refactoring

대규모 리팩토링을 한 번에 수행하지 않고, **작은 단계**로 나누어 각 단계마다 main 브랜치에 병합하여 충돌을 방지합니다.

```
userValidation.js (Evolution)

// Step 1: 함수 추출 (Day 1 - Main 병합)
function validateUser(user) {
  return user.name && user.email;
}

// Step 2: JSDoc 타입 추가 (Day 2 - Main 병합)
/** @param {User} user */
function validateUser(user) {
  return user.name && user.email;
}

// Step 3: 에러 처리 강화 (Day 3 - Main 병합)
function validateUser(user) {
  if (!user.name) throw new Error('Name required');
  if (!user.email) throw new Error('Email required');
  return true;
}
```




SECTION 07

TBD 체크리스트

준비도와 일일 습관

TBD 체크리스트



성공적인 트렁크 기반 개발을 위한 조직적 준비와 개인의 실천 사항

팀 준비도 (Organizational Readiness)

- ☒ CI/CD 파이프라인 구축 (자동화된 배포)
- ☒ 자동화된 테스트 (단위, 통합 테스트 커버리지)
- ☒ Feature Flags 시스템 도입
- ☒ 빠른 빌드 속도 유지 (10분 이내 권장)
- ☒ 효율적인 코드 리뷰 프로세스
- ☒ 엄격한 브랜치 보호 규칙 (Branch Protection)
- ☒ 실시간 모니터링 및 알림 설정

일일 실천 사항 (Daily Routine)

- ☒ 항상 최신 main 브랜치에서 작업 시작
- ☒ 작은 단위의 커밋 생성 (2-4시간 분량)
- ☒ 하루에 최소 1회 이상 main에 Push/Merge
- ☒ 요청받은 코드 리뷰는 2시간 이내 처리
- ☒ CI 테스트 통과 시 즉시 병합
- ☒ 병합 완료된 로컬/원격 브랜치 즉시 삭제



SECTION 08

GitHub 설정

보호 규칙과 워크플로우



GitHub 설정: Branch Protection & Actions

안정적인 TBD 운영을 위해 브랜치 보호 규칙과 자동화된 워크플로우를 설정합니다.



Branch Protection Rules

```
# main 브랜치 보호 설정
branch_protection_rule:
  pattern: "main"
  # 1. 코드 리뷰 필수 (1명 이상)
  required_pull_request_reviews:
    required_approving_review_count: 1
  # 2. CI 테스트 통과 필수
  required_status_checks:
    strict: true # 최신 상태 유지
    contexts: ["test", "lint", "build"]
  # 3. 선형 히스토리 (Merge Commit 방지)
  require_linear_history: true
  # 4. 병합 후 브랜치 자동 삭제
  delete_head_branches: true
```



TBD Automation Workflow

```
.github/workflows/trunk.yml

name: Trunk-Based CI
on: { pull_request: { branches: [main] } }
jobs:
  validate-tbd-practices:
    runs-on: ubuntu-latest
    steps:
      # 1. 브랜치 나이 체크 (24시간 제한)
      - name: Check branch age
        run: |
          created=$(git log --format=%ct origin/${{ head }} | tail -1)
          age=$(( (${date +%s} - $created) / 3600 ))
          if [ $age -gt 24 ]; then exit 1; fi

      # 2. 커밋 크기 체크 (500줄 제한)
      - name: Check PR size
        run: |
          lines=$(git diff --shortstat origin/main | awk '{print $4+$6}')
          if [ $lines -gt 500 ]; then exit 1; fi

      # 3. 테스트 실행
      - run: npm test && npm run lint
```



SECTION 09

TBD 메트릭

측정과 시각화



09

TBD 메트릭 측정



개발 생산성을 수치화하여 **Health Score**로 관리합니다.

```
metrics.js

const tbdMetrics = {
  avgBranchAge: 8, // hours (Target: < 24h)
  dailyCommits: 15, // per dev
  avgPRSize: 250, // lines (Target: < 500)
  mergesPerDay: 12 // frequency
};

function calculateHealthScore(metrics) {
  let score = 100;
  // 1. 브랜치 수명 (-20)
  if (metrics.avgBranchAge > 24) score -= 20;
  // 2. PR 크기 (-20)
  if (metrics.avgPRSize > 500) score -= 20;
  // 3. 병합 빈도 (-20)
  if (metrics.mergesPerDay < 10) score -= 20;
  return score;
}

console.log(`TBD Score:
${calculateHealthScore(tbdMetrics)}`);
```

KEY PERFORMANCE INDICATORS (KPIs)



브랜치 수명 (Branch Age)

브랜치가 생성된 시점부터 병합될 때까지의 시간

< 24h

TARGET



일일 커밋 수 (Commit Frequency)

개발자가 하루에 메인 브랜치로 푸시하는 횟수

High

Goal



PR 크기 (Lines of Code)

한 번의 풀 리퀘스트에 포함된 변경 라인 수

< 500

LINES



병합 빈도 (Merge Rate)

팀 전체가 하루에 수행하는 병합 횟수

> Team Size

MIN



SECTION 10

안티패턴

피해야 할 습관



10

안티패턴: 피해야 할 습관



장기 Feature 브랜치

문제점: 며칠·몇 주간 브랜치를 유지하면 메인과 격차가 커져 심각한 병합 충돌 발생.

✨ **해결책:** 하루 단위로 작업 쪼개기

Feature Flag 미정리

문제점: 배포 후 플래그를 방치하면 '좀비 코드'가 되어 유지보수를 방해하는 기술 부채화.

✨ **해결책:** 100% 롤아웃 후 즉시 삭제

거대한 PR (Mega PR)

문제점: 수천 줄의 코드는 리뷰가 불가능하며 버그 발견이 어렵고 병목을 유발.

✨ **해결책:** < 500줄 단위로 분할

anti_patterns_vs_best_practices.sh

1. Branch Lifecycle Strategy

❌ **BAD** Long-lived (2 weeks)
git checkout -b feature/big-refactor
... 2 weeks of coding ...
git push origin feature/big-refactor
CONFLICT: 156 files changed

✅ **GOOD** Incremental (Daily)
git checkout -b refactor-step-1
... 4 hours work ...
git push origin refactor-step-1
MERGED: Clean & Fast

----- SCENARIO 2 -----

2. Managing Technical Debt (JavaScript)

❌ **BAD** if (flags.oldFeature2020) { /* Dead code */ }
✅ **GOOD** // After 100% rollout: Remove flag & old code
function processData(data) {
 // No flag check needed anymore
 return newImplementation(data);
}

----- SCENARIO 3 -----

3. Pull Request Size (Git Stats)

❌ **BAD** Review Impossible
commit: "Rewrite entire app"
+5,240 insertions, -3,100 deletions
Review Time: 5 days

✅ **GOOD** Reviewable
commit: "Step 1: Add new models"
+50 insertions, -0 deletions
Review Time: 30 mins



SECTION 11

TBD 마이그레이션 가이드

Git Flow → TBD



TBD 마이그레이션 가이드



WEEK 1

Phase 1: Foundation

기반 구축 단계입니다. CI/CD 파이프라인을 검증하고 Feature Flag 시스템을 도입하며, 팀원 교육을 진행합니다.



WEEK 2-3

Phase 2: Hybrid Practice

하이브리드 운영 기간입니다. develop 브랜치를 유지하되 브랜치 수명을 3일 → 1일로 점진적으로 단축합니다.



WEEK 4+

Phase 3: Full TBD

완전한 전환 단계입니다. develop 브랜치를 삭제하고 main 브랜치만 사용하며 모든 변경사항을 1일 내 병합합니다.

migration_checklist.md

TBD 마이그레이션 실행 체크리스트

Phase 1: Foundation (준비)

- ☒ CI/CD 파이프라인 검증 (빌드 시간 < 10분)
- ☐ Feature Flags 도입 (LaunchDarkly/Unleash)
- ☐ 팀 교육 (TBD 개념 및 실습 세션 진행)

Phase 2: Practice (적응)

- ☐ 브랜치 생명주기 단축 (7일 → 3일 → 1일)
- ☐ PR 크기 축소 목표 설정 (< 500 lines)
- ☐ 병합 빈도 증가 (주 1회 → 일 1회 이상)

Phase 3: Full TBD (완료)

- ☐ develop 브랜치 삭제 및 아카이빙
- ☐ main 브랜치만 단일 소스로 사용
- ☐ TBD 메트릭(브랜치 수명, 커밋 수) 모니터링 대시보드
- ☐ 지속적 개선 회고 진행



SECTION 12

실습 과제

Hands-on



실습 과제 (Hands-on Labs)



Trunk-Based Development의 핵심 프로세스를 직접 경험하고 환경을 구축합니다.

Mission 01



TBD 워크플로우 실천

- 반드시 **main 브랜치**에서 작업을 시작합니다.
- 2-3시간 분량의 작은 기능을 구현합니다.
- PR 생성 및 동료 리뷰 후 즉시 병합합니다.
- 위 과정을 하루 동안 최소 **3회** 반복합니다.

Mission 02



Feature Flags 도입

- 간단한 Feature Flag 시스템을 구현하거나 라이브러리를 설치합니다.
- 미완성 기능을 Flag로 감싸서 **사용자에게 숨깁니다**.
- 기능이 완성되지 않았더라도 코드를 main에 병합합니다.
- 개발 완료 후 Flag를 활성화하여 배포합니다.

Mission 03



Metrics Dashboard

- 팀의 TBD 건강도를 측정할 **핵심 지표**를 정의합니다.
- 브랜치 수명, PR 크기, 일일 병합 빈도를 수집합니다.
- 간단한 대시보드를 구축하여 팀원들과 공유합니다.
- 데이터에 기반한 개선 목표(예: 브랜치 수명 < 1일)를 설정합니다.

Mission 04



CI/CD 최적화

- GitHub Actions 워크플로우를 작성합니다.
- **브랜치 나이 체크** 스크립트를 CI 파이프라인에 추가합니다.
- PR 크기가 특정 라인 수(예: 500줄)를 넘으면 경고를 띄웁니다.
- (선택) Dependabot 등 봇의 PR은 테스트 통과 시 자동 병합합니다.



SECTION 13

참고 자료

필수 읽기/도구/블로그



References & Resources

Trunk-Based Development와 관련된 심화 학습 자료, 추천 도구 및 유용한 블로그 포스트 모음입니다.



MUST READ

필수 읽기 자료



Trunk Based Development 

TBD의 정의, 스타일, 장단점을 망라한 공식 가이드 웹사이트입니다.



Google's Approach to TBD 

Google 엔지니어링 팀이 어떻게 거대한 단일 저장소에서 TBD를 실천하는지 보여주는 영상입니다.



Accelerate Book 

소프트웨어 전달 성과와 TBD의 상관관계를 과학적으로 분석한 필독서입니다.



TOOLS

추천 도구 (Feature Flags)



LaunchDarkly 

엔터프라이즈급 기능 플래그 관리 플랫폼으로, 점진적 롤아웃을 지원합니다.



Unleash 

오픈소스 기반의 기능 토글 시스템으로, 직접 호스팅하여 사용할 수 있습니다.



Backstage 

Spotify에서 개발한 개발자 포털로, 서비스 카탈로그와 문서화를 통합 관리합니다.



ARTICLES

기술 블로그



Atlassian - TBD vs Git Flow 

CI/CD 관점에서 두 브랜치 전략의 차이점과 장단점을 명확하게 비교한 아티클입니다.



ThoughtWorks - Feature Toggles 

Martin Fowler의 블로그로, Feature Toggle의 다양한 패턴과 구현 방식을 깊이 있게 다룹니다.



DevOps Research (DORA) 

고성과 조직의 4가지 핵심 지표와 TBD의 연관성에 대한 연구 결과입니다.

 Prof. Sejin Chun

AI Open Source Software

30



SECTION 14

핵심 요약

Key Takeaways

Key Takeaways



Trunk-Based Development의 5가지 핵심 성공 요소를 기억하세요.



01

단일 브랜치 (Single Branch)

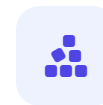
복잡한 브랜치 전략 대신 **main** 브랜치 하나에 집중하세요. develop, release 등 장기 브랜치를 제거하여 통합의 복잡성을 줄입니다.



02

짧은 생명주기

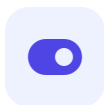
모든 피쳐 브랜치의 수명은 **24시간 이내**여야 합니다. 짧은 주기는 병합 충돌을 예방하고 코드의 신선도를 유지합니다.



03

작은 커밋 & 자주 통합

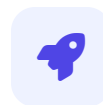
"Commit early, commit often." 거대한 변경사항을 작은 단위로 쪼개어 **하루에 여러 번** main에 통합하세요.



04

Feature Flags

배포와 출시를 분리하세요. 미완성 기능은 Flag로 숨겨 안전하게 main에 병합하고 프로덕션에 배포할 수 있습니다.



05

빠른 피드백 (CI/CD)

강력한 **자동화 테스트**와 CI 파이프라인이 필수입니다. 통합 즉시 코드의 안정성을 검증하고 피드백을 받으세요.

TBD vs Git Flow 비교표



전통적인 브랜치 전략(Git Flow)과 현대적인 트렁크 기반 개발(TBD)의 주요 차이점

비교 항목	 Git Flow	 Trunk-Based Development
 브랜치 수	많음 (Develop, Release, Feature, Hotfix)	적음 (Main + Short-lived Feature)
 브랜치 수명	장기 (수일 ~ 수주)	단기 (24시간 이내 권장)
 통합 빈도	낮음 (기능 완성 시점)	매우 높음 (하루에도 수십 번)
 병합 충돌	빈번함 / 복잡함 (Merge Hell 위험)	드물 / 간단함 (즉시 해결 가능)
 배포 속도	느림 (릴리스 주기에 종속)	빠름 (Continuous Delivery 최적화)
 학습 곡선	가파름 (엄격한 규칙 이해 필요)	완만함 (단순한 워크플로우)
 관리 복잡도	높음 (여러 브랜치 동기화 필요)	낮음 (Main 브랜치만 관리)

Next Steps & Conclusion



TBD 도입을 위한 구체적인 실행 계획과 마지막 제언



01

TBD 실천 시작

오늘부터 **main** 브랜치에 작은 변경사항을 병합해 보세요.
Feature Flags를 활용하면 미완성 기능도 안전하게 통합할 수 있습니다.



02

메트릭 모니터링

브랜치 수명(24h 미만)과 병합 빈도를 측정하세요. **데이터 기반**으로 병목 구간을 찾고 팀의 개발 속도를 개선할 수 있습니다.



03

지속적 개선

팀 회고를 통해 코드 리뷰 속도와 CI 파이프라인 성능을 점검하세요. 도구 도입보다 중요한 것은 **협업 문화**의 정착입니다.

“

**"Commit early, commit often,
to main."**

작게 나누고, 자주 통합하여, 더 빠르게 배포하세요.

Next Week Preview



다음 주차에는 Trunk-Based Development를 기반으로 한 **고급 배포 전략**과 **DevOps 문화**를 심도 있게 다룰 예정입니다.



WEEK 12 TOPIC

Advanced CI/CD & DevOps

TBD 환경에서의 자동화된 배포 파이프라인 구축, Blue-Green 배포 전략, 그리고 DevOps 문화의 정착에 대해 학습합니다.



PREPARATION

실습 환경 점검

오늘 실습한 GitHub Actions 워크플로우가 정상 작동하는지 확인하고, 개인 AWS 또는 클라우드 계정을 미리 점검해 주세요.



PRE-READING

Google SRE Book

Google SRE Book의 "Release Engineering" 챕터를 읽어오시면 배포 전략 이해에 큰 도움이 됩니다. (LMS 링크 참조)



NOTICE

팀 프로젝트 중간 점검

다음 주 수업 종료 후 팀별 프로젝트 진행 상황 1차 리뷰가 있을 예정입니다. 이슈 트래커를 최신화해 주세요.